

SPS Modelling Competition

Prompt 1: Routing on Curved Worlds

By Brian, James, Shresht, Aiden

Understanding the Problem

Requirements, Constraints, Score Maximisation

50 destinations

1,000 tokens per world

Choose the endpoint, vehicle, load, and checkpoints.

To maximise Score: Maximize demand-weighted delivery quality.

Deconstructing the Score Function

$$\rho = D/D_0 \geq 1$$

$$W(\rho) = e^{-0.20(\rho-1)}$$

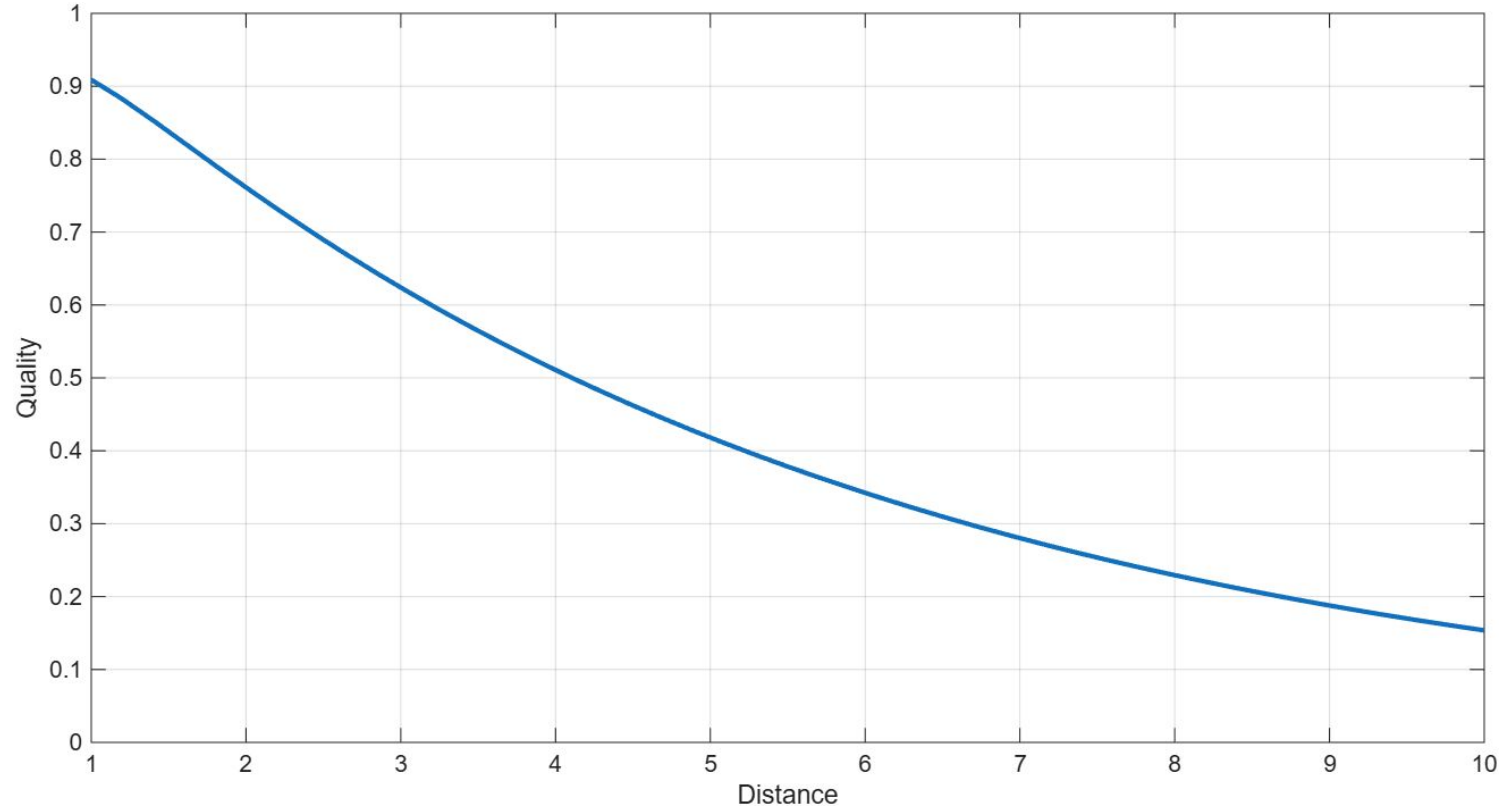
$$p_{\text{attack}} = 1 - \exp[-0.10 t e^{-3(\rho-1)}]$$

$$q_j = \min\left(\frac{L_j}{D_j}, 1\right) (0.95)^t e^{-0.02t} (1 - 0.25 p_{\text{attack}}) W(\rho)$$

Deconstructing Global Score Function

$$C_M = 100 \frac{\sum_j D_j q_j}{\sum_j D_j} \left[0.85 + 0.15 \left(1 - \min \left(\frac{F_{tok}}{1000}, 1 \right) \right) \right]$$

Quality of Delivery Versus Distance of Delivery



```

def get_base_line(self) -> None:
    for world_type in WORLD_TYPES:
        # Constants
        total_tokens = 1000
        world = self.instances[world_type]
        d_index = []

        # Sort by shortest distance
        for i, pt in enumerate(world.endpoints):
            distance = geodesic_distance(world, world.start, pt) # type: ignore
            d_index.append((distance, i))
        d_index = sorted(d_index)

        # Greedily allocate the horse/truck/jet for small distances
        all_deliveries: list[DeliveryPath] = []
        for distance, i in d_index:
            if distance < 50 * 1000:
                vehicle = "horse"
            elif distance < 750 * 1000:
                vehicle = "truck"
            else:
                vehicle = "jet"

            # evaluate potential delivery method
            delivery = evaluate_delivery(world, i, vehicle, ())
            if total_tokens - delivery.total_tokens > 0:
                total_tokens -= delivery.total_tokens
                all_deliveries.append(DeliveryPath(i, vehicle, []))

        self.paths[world_type] = all_deliveries

```

```

def get_base_line(self) -> None:
    for world_type in WORLD_TYPES:
        # Constants
        total_tokens = 1000
        world = self.instances[world_type]
        d_index = []
        # Sort by shortest distance
        for i, pt in enumerate(world.endpoints):
            distance = geodesic_distance(world, world.start, pt) # type: ignore
            d_index.append((distance, i))
        d_index = sorted(d_index)

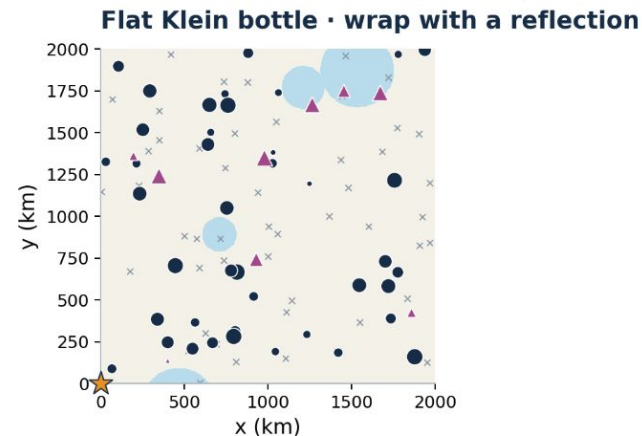
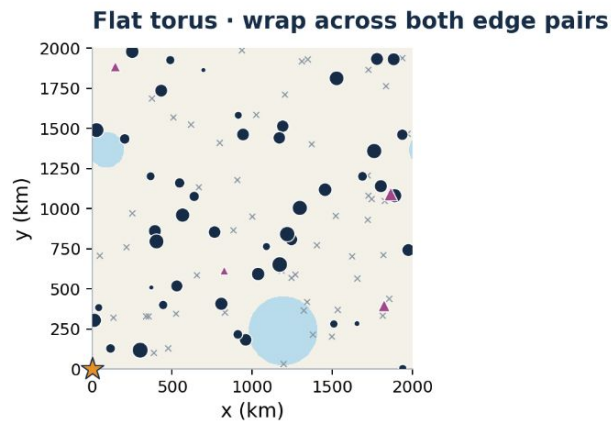
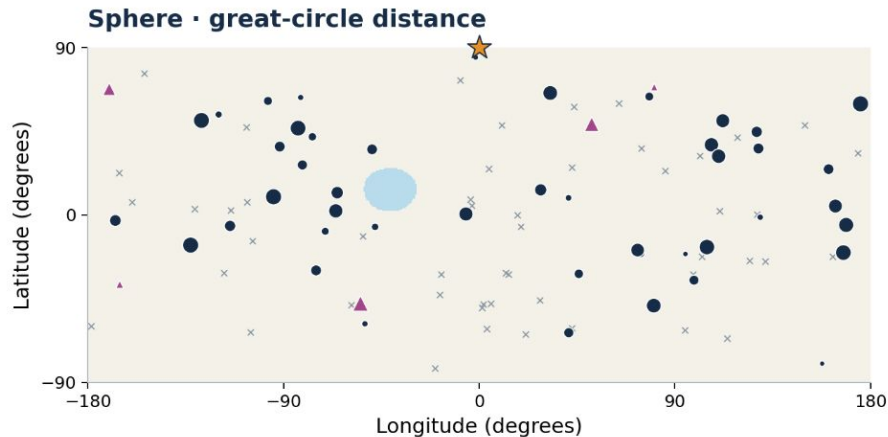
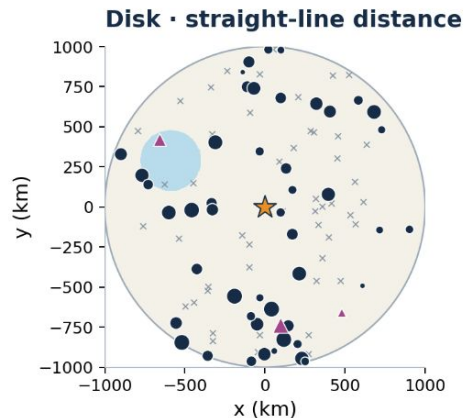
        # Greedily allocate the horse/truck/jet for small distances
        all_deliveries: list[DeliveryPath] = []
        for distance, i in d_index:
            if distance < 50 * 1000:
                vehicle = "horse"
            elif distance < 750 * 1000:
                vehicle = "truck"
            else:
                vehicle = "jet"
            # evaluate potential delivery method
            delivery = evaluate_delivery(world, i, vehicle, ())
            if total_tokens - delivery.total_tokens > 0:
                total_tokens -= delivery.total_tokens
                all_deliveries.append(DeliveryPath(i, vehicle, []))
        self.paths[world_type] = all_deliveries

```

Seed	Score
67	16.041
42	13.625
420	15.715

Seed	Score
67	20.4
42	N/A
420	N/A

Seed 67 Visualization



★ Depot ● Destination ▲ Island flag (jet only) × Checkpoint ■ Ocean region

Baseline

A simple reference method for measuring improvement.

- 1 Sort destinations by direct distance**
- 2 Choose a vehicle using distance thresholds**
- 3 Accept feasible deliveries while budget remains**

Direct routes only; no checkpoint search

Precompute ocean crossings

Reuse each ocean test throughout the route search.

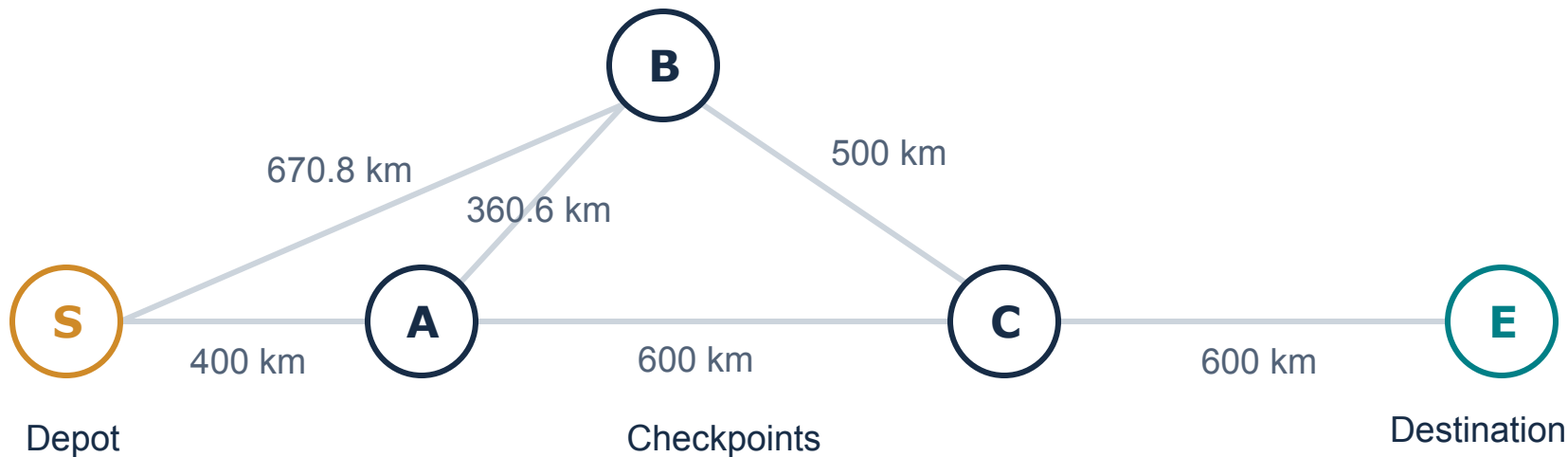
Connection type	Tests per world
Depot to checkpoints / endpoints	100
Checkpoint to checkpoint	1,225
Checkpoint to endpoint	2,500

3,825 cached ocean tests per world

Use the reference ocean test; apply the vehicle's range limit.

Ground routes graph

Illustrative truck graph with no oceans. Distances in km; diagram not to scale.



Truck: maximum leg 750 km

The state includes the number of stops

$M_n(i)$: shortest distance to checkpoint i with exactly n checkpoint visits.

$M_1(i) = d(S, i)$ for a permitted depot leg

$$M_n(i) = \min \{ M_{n-1}(j) + d(j, i) \}$$

Only permitted edges $j \rightarrow i$ enter the minimum.

$n - 1$ checkpoint visits $\rightarrow$ one more permitted leg $\rightarrow$ n visits

One update chooses the best predecessor

Example: reaching C with exactly two checkpoint visits.

Predecessor	Distance so far	Final leg	Total
A	400 km	600 km	1,000 km
B	670.8 km	500 km	1,170.8 km

$M_2(C) = 1,000$ km

$\text{traceback}_2(C) = A$

Store the distance
and the predecessor.

Our Solution Idea

```
for i, endpoint in enumerate(endpoints):
    if direct_endpoint_distances[i] <= vehicle.max_leg_m and not
start_endpoint_crosses_ocean[i]:
        candidate_path_distances[0][i] = direct_endpoint_distances[i]

    for n in range(1, N_CHECKPOINTS + 1):
        for j, midpoint in enumerate(checkpoints):
            if candidate_midpoint_distances[n][j] == math.inf or
midpoint_endpoint_crosses_ocean[j][i]:
                continue

            leg = geodesic_distance(instance, midpoint, endpoint)
            new_distance = candidate_midpoint_distances[n][j] + leg
            if leg <= vehicle.max_leg_m and new_distance <=
candidate_path_distances[n][i]:
                candidate_path_distances[n][i] = new_distance
                candidate_path_traceback[n][i] = j
```

candidate_midpoint_distances[n][j]			
	M0	M1	M2
n=0	—	—	—
n=1	30	45	∞
n=2	60	70	55
n=3	95	90	80

leg to endpoint (max_leg_m = 50)			
	E0	E1	E2
start	40	120	45 ocean
M0	20	35	30 ocean
M1	15	25	40
M2	30	55	20

candidate_path_distances[n][i]			
	E0	E1	E2
n=0	00	00	00
n=1	00	00	00
n=2	00	00	00
n=3	00	00	00

ready

step 0 / 30

The table fills one endpoint column at a time, because the n and j loops sit inside the loop over i.

Using a DP approach, we can store locally optimal paths allowing us to efficiently compute further optimal solutions from the stored values (animation generated by Claude).

Attach the endpoint, then trace backward

$P_{\square}(e) = \min_{\square}(M_{\square}(j) + d(j, e))$ over permitted final legs.



Selected route: S → A → C → E

2 checkpoint visits

Total distance: 1,600 km

Evaluate each candidate delivery

The reference scorer supplies feasibility, quality, and token cost.

$$\text{Load} = \min(\text{demand}, \text{vehicle capacity})$$

$$\text{Benefit} = \text{demand} \times \text{delivery quality}$$

$$\text{Cost} = \text{dispatch} + \text{fuel}$$

Candidates: truck routes, horse routes, and direct jets

Greedy selection by benefit per token

Rank all candidates, then build a budget-feasible plan.

$$\text{Priority} = \text{demand} \times \text{quality} / \text{token cost}$$

- 1 Visit options in descending priority
- 2 Choose at most one option per endpoint
- 3 Accept only if the full plan score improves

Local refinement improves the selected plan

Try changing one endpoint's choice at a time.

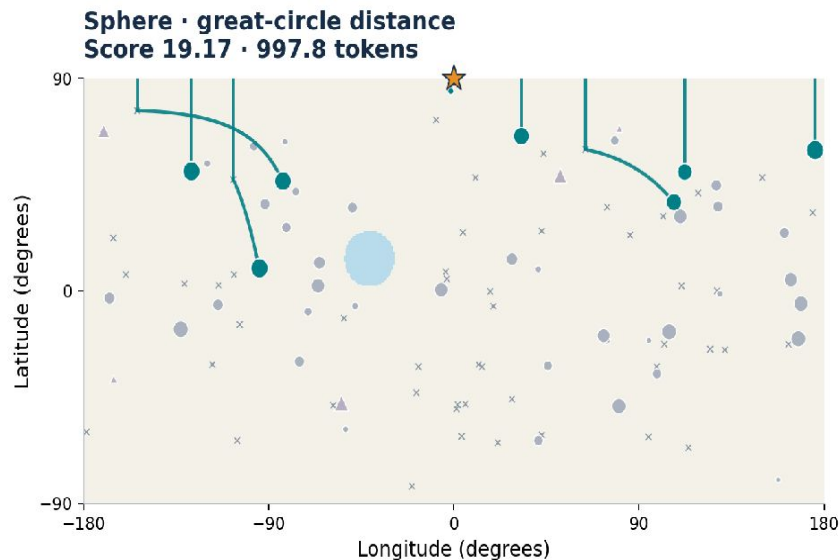
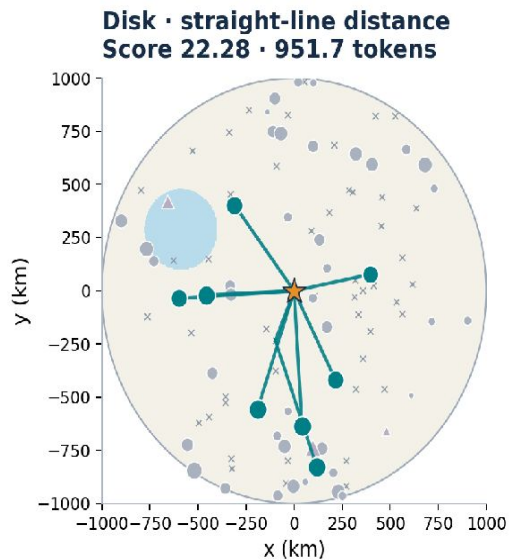
Current choice

Try another route, another vehicle, or no delivery

Keep the change only when the total score increases

Up to 3 passes; stop early when nothing changes

Selected routes: disk and sphere



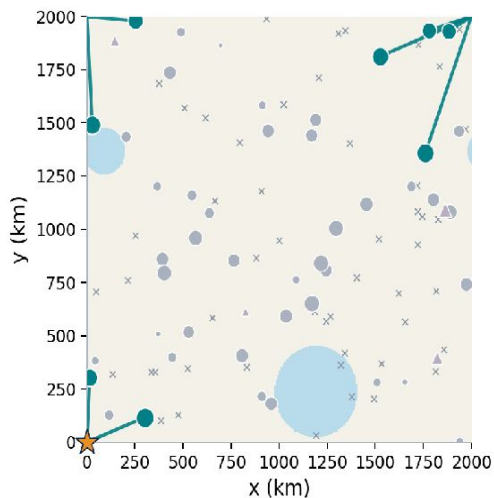
North-pole depot: all points along the top edge represent the same location.

Truck Jet Horse ★ Depot Ocean

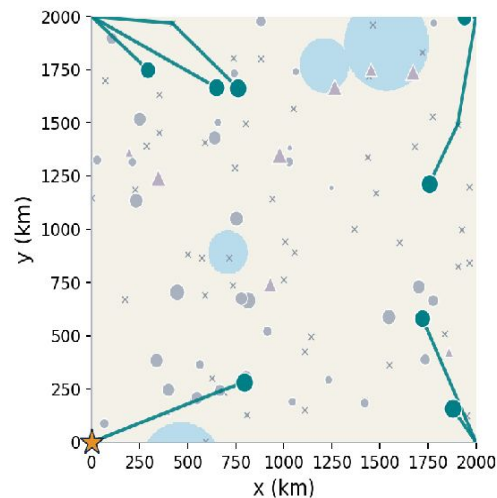
Coloured endpoints receive deliveries; grey endpoints are unserved. Triangles mark islands. Marker area scales with demand.

Selected routes: torus and Klein bottle

Flat torus · wrap across both edge pairs
Score 20.87 · 898.3 tokens



Flat Klein bottle · wrap with a reflection
Score 20.24 · 952.5 tokens



Truck Jet Horse ★ Depot Ocean

Coloured endpoints receive deliveries; grey endpoints are unserved. Triangles mark islands. Marker area scales with demand.

Seed 67: comparison with the baseline

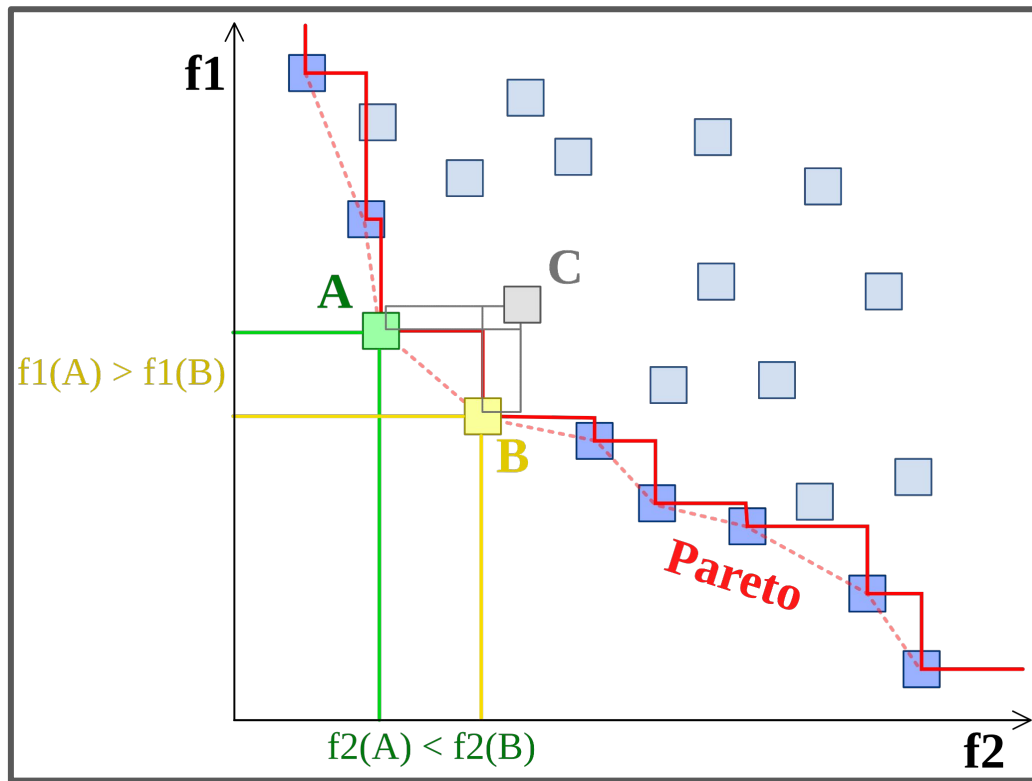
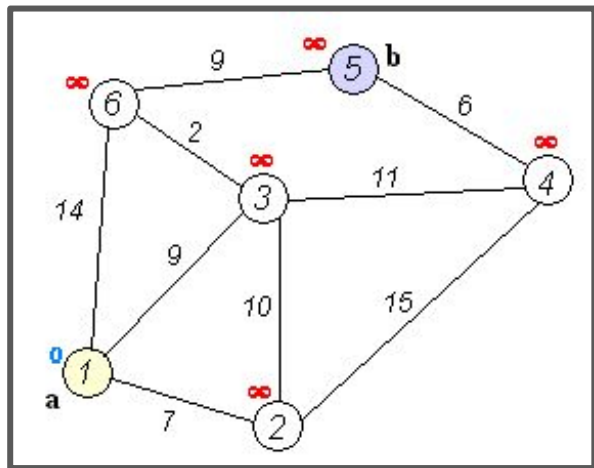
Scores from the official scorer, using the returned delivery plans.

World	Baseline	New method
Disk	16.22	22.28
Sphere	12.96	19.17
Torus	16.19	20.87
Klein	17.15	20.24

Weighted score: 16.04 → 20.40

27.2% improvement on the public instance

Dijkstra's Algorithm



AI use and reproducibility

Confirm the contribution statement before recording.

ChatGPT: figures, slides, code review, and score checks

Claude: algorithm animation

Checks: numerical examples and reference-scored plans